TECHNICAL OVERVIEW

An AI infrastructure for product companies

Three things, working together: keep a product's knowledge alive with AI instead of losing it to turnover and stale docs; turn the handoff between product, design, and engineering into an explicit, enforceable contract; and let dozens of coding agents work in parallel on that contract without corrupting each other's work.

---

01 the problem

Running one agent is easy. Running fifty is a distributed-systems problem.

Point a single coding agent at a repo and it behaves like a fast, tireless engineer. Point fifty at the same task list and a different set of failures shows up — not "the AI wrote bad code," but the process itself silently breaking: two agents claim the same task, a reviewer gets dispatched twice onto one pull request, a container dies mid-task and the work vanishes with it.

A concrete version of this: a task finishes implementation and moves into review. If "has a reviewer already been dispatched for this task" is a question the system answers by *checking* state and then separately *acting* on it, two reviewer agents can both check at the same instant, both see "no," and both start — now two independent verdicts land on one pull request. The fix isn't a smarter agent. It's making the check-and-claim a single, indivisible operation the moment it happens, so only one of the two ever succeeds.

> THE PATTERN

> Almost every hard problem in this system looks like an AI problem and turns out to be a concurrency problem wearing an AI costume. Solving it is what makes "fifty agents coding at once" a safe sentence instead of a scary one.

---

02 positioning

Not a smarter autocomplete. Not a black box you rent by the prompt.

Three categories already claim to do "AI coding." None of them solve the problem above, because none of them were built to run more than one agent against the same shared state at a time.

CATEGORY	WHAT IT'S GOOD AT	WHERE IT BREAKS AT FLEET SCALE
Single-agent coding assistants Cursor, Copilot-style tools	One developer, one file, one sitting — excellent second pair of hands	No concept of a task queue, a claim, or a review gate — there's no "fleet" to coordinate
Hosted background-agent products	Convenient — no infrastructure to run	The runtime is theirs, not yours: no deploying inside your network, no holding your own credentials, no inspecting the audit trail
A task list plus a chat bot	Looks automated from the outside	State lives wherever the last script left it — a crash mid-task quietly loses work, with nothing recorded about why
This platform	**Runs inside your infrastructure; every claim on shared work is atomic; every stall has a recorded reason and a human unblock path; agent credentials never sit with the piece of the system that decides what work to hand out.**	

---

03 workflow

The contract between departments, made explicit

Every product company already runs this handoff — product decides what to build, a tech lead decides how, engineers build it, someone reviews it. Usually that contract lives in

people's heads and Slack threads, re-negotiated every time. Here it's a schema, with two layers.

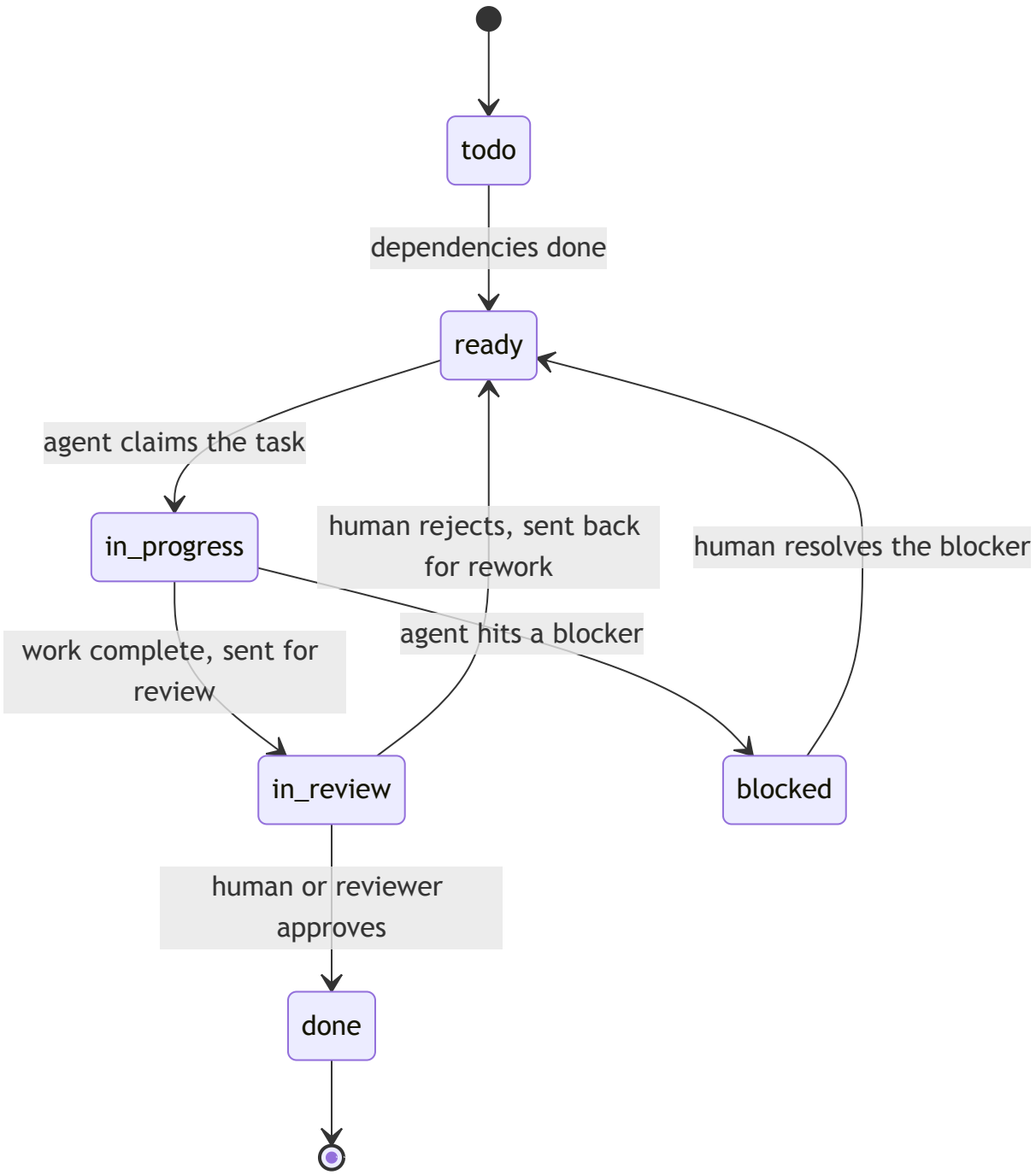
Feature layer — the department handoff

One feature moves through a fixed sequence of owners, and every handover between them is a human approval gate, not an assumption someone made in a hallway.

Every arrow is a human approval gate — not an implicit assumption.

Task layer — the unit the concurrency guarantee is built on

Underneath one feature, work is broken into tasks: one task, one repository, an explicit list of what it depends on, and a status that only one kind of actor is ever allowed to move at each step.

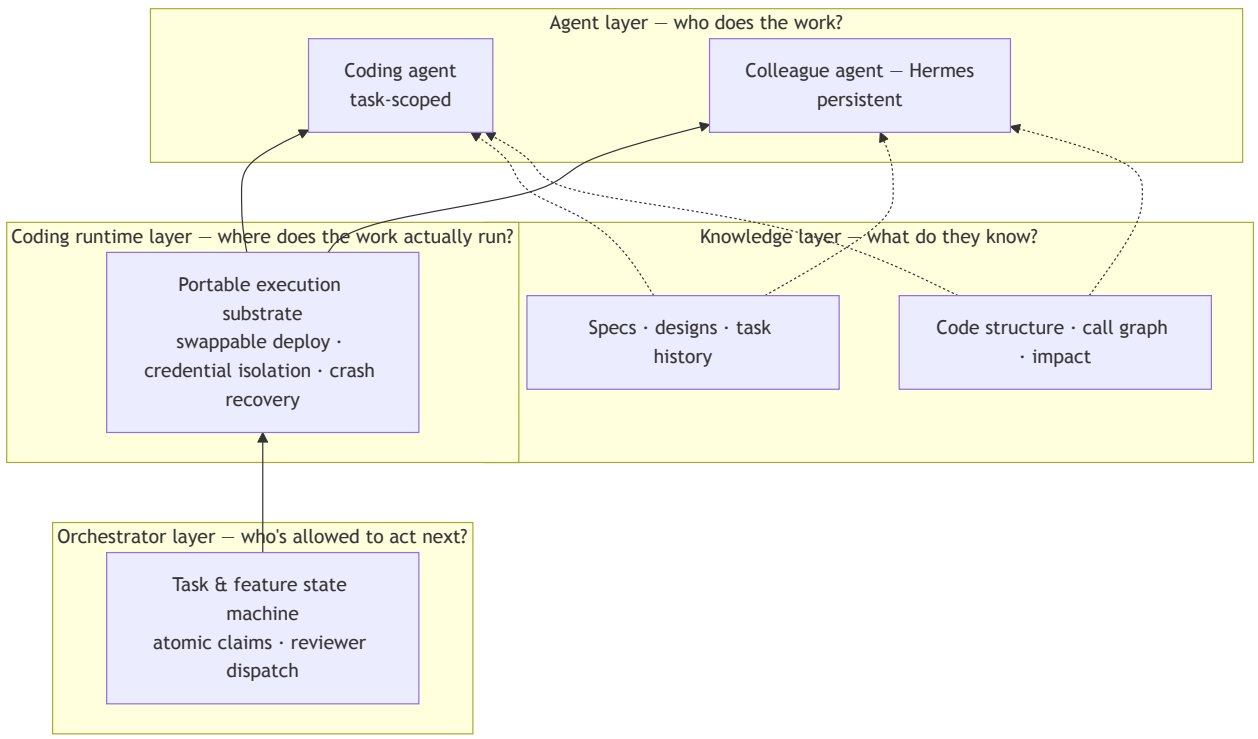


Exactly one actor is allowed to move each edge — agent, orchestrator, or human — never "whoever gets to it first."

---

04 architecture

Four layers, each answering one question



Solid = execution flow, bottom to top. Dashed = knowledge available to any running agent.

Orchestrator layer

Owns the state machine from §03 and the claim protocol underneath it. This is what makes "fifty agents, zero corruption" literally true: claiming a task, dispatching a reviewer, and marking work done are each a single indivisible operation — not a sequence of checks an agent could get caught in the middle of.

Coding runtime layer

The substrate that actually runs an agent against a real repository, wherever your infrastructure lives — locally, in your own network, or as a fleet of containers. Credentials that can push code never sit with the part of the system that decides *what* to run — only with the part that runs it.

Knowledge layer

Keeps specs, designs, decisions, and code structure searchable by agents and humans alike, isolated per company. An agent asks "what does this feature mean" the same way a new hire would ask a teammate — instead of re-reading the whole codebase from scratch on every task.

Agent layer — two different kinds of agent

TASK-SCOPED

Coding agent

Claims a task, writes the code, opens the pull request — and disappears when it's done. It exists entirely inside the contract from §03.

PERSISTENT

Colleague agent — Hermes

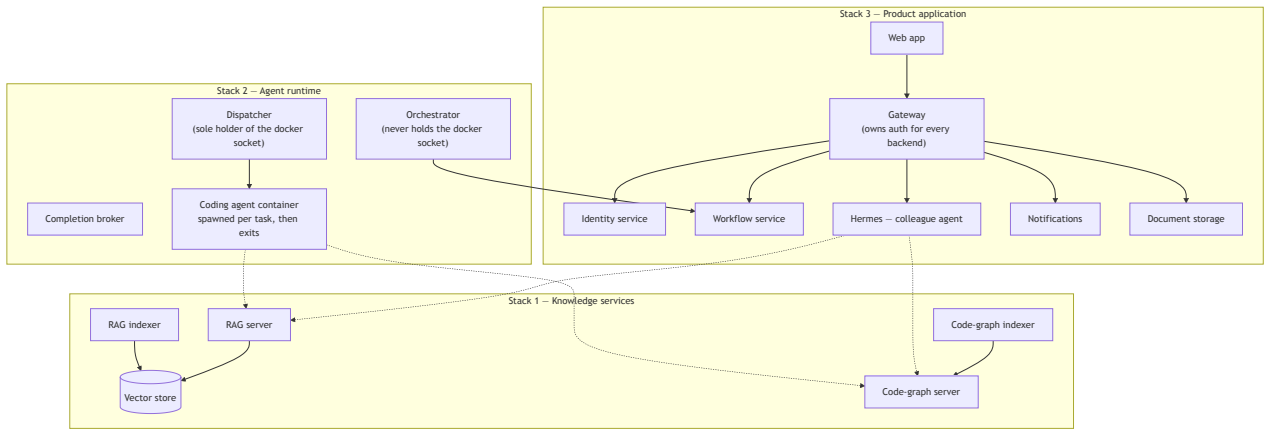
Not task-scoped. Participates in chat, remembers context across sessions, and adapts its own skills over time — closer to hiring a colleague than installing a tool.

---

05 deployment

Three stacks, one shared network, three different owners

In production this isn't one monolith — it's three independently deployable stacks on a shared network, because each one has a different reason to exist and a different reason to scale: the knowledge layer is long-lived shared infrastructure, the agent runtime is an ephemeral worker fleet that grows and shrinks with task volume, and the product application is the one thing a human ever opens in a browser.



Dashed edges are knowledge-layer queries; solid edges are the request/task path.

3

independently deployable stacks

1

container holds the spawn privilege

0

credentials touch the orchestrator

That last number is the coding runtime layer's credential-isolation claim, made concrete: across the entire deployment, exactly one container ever holds the privilege to start new work — the dispatcher. The orchestrator that decides what should run next never touches it, and coding-agent containers exist only for the lifetime of the task that spawned them.

The gateway in Stack 3 is the only thing a browser ever talks to — it owns authentication once, centrally, so no backend service reimplements login, sessions, or access control on its own.